SmartBudget ML

Application mobile de budget personnel 100 % locale pour le Cameroun (FCFA, français / anglais) — Rapport de clôture, Activité 8

Auteur : LISSOM ELOGE DEKENZO ABEL **Dépôt GitHub :** https://github.com/ElogeKenzo/SmartBudgetML

Date : octobre 2026

1. Objectifs du projet

Construire une application Android de gestion de budget adaptée au contexte camerounais, qui fonctionne **sans connexion Internet** et sans envoyer aucune donnée hors du téléphone.

- **Suivi des dépenses et revenus** en FCFA, avec 17 catégories camerounaises (marché, transport moto/taxi, ENEO/Camwater, tontine, solidarité familiale, transferts MoMo/OM…).
- **Catégorisation automatique** des dépenses par apprentissage automatique embarqué (ML.NET) : mémoire des corrections de l'utilisateur, puis règles par mots-clés, puis modèle.
- **Prévision de fin de période** (régression linéaire) avec indice de fiabilité, et budget par **enveloppe hebdomadaire** pour les revenus irréguliers.
- **Tontines / njangi** : membres, ordre de passage, cotisations, pot reçu, retards, rappel « À cotiser ».
- **Sécurité** : base chiffrée SQLCipher, clé dans le coffre du système, code PIN à 6 chiffres (PBKDF2), verrouillage automatique, biométrie.
- **Données** : export CSV, sauvegarde chiffrée par mot de passe et restauration sur un autre téléphone.

Technologies : .NET 10 MAUI Blazor Hybrid (C#), SQLite chiffré (SQLCipher), ML.NET, xUnit, GitHub Actions avec test automatique sur émulateur Android. Seule permission Android : `USE_BIOMETRIC` (pas d'accès aux SMS ni à Internet).

2. Démarche

Le projet a avancé par phases (0 à 7) puis par une **phase 8 de correction**, après un audit du code qui a montré que certaines fonctions affichées n'étaient pas réelles (maquettes, valeurs codées en dur). Règle adoptée : une fonction n'est annoncée que si elle est atteignable depuis l'écran, enregistrée en base et prouvée par un test. Chaque lot suit GitFlow (`feature/…` depuis `develop`, fusion et étiquette `Phase8-…`), et un script unique (`tools/verify.ps1`) lance les tests, le build Android et des garde-fous automatiques.

3. Taux de réalisation

Mesuré sur le registre d'audit (`docs/AUDIT_REGISTER.md`), qui recense 41 constats : 24 de l'audit initial (A1–A24) et 17 de l'audit de phase 8 (N1–N17).

Statut des constats	Nombre	Part
Corrigés et prouvés (FIXED)	33	80,5 %
Partiels (PARTIEL)	4	9,8 %
Restant à traiter (OUVERT)	4	9,8 %

Lots de la phase 8	État
8A Build réel et vérification · 8B CI et test sur émulateur · 8C Sécurité (verrou, clé, PIN) · 8D Export, sauvegarde, restauration · 8E Tontines et revenus irréguliers	**Terminés (5 / 8, soit 62,5 %)**
8F Import des SMS Mobile Money · 8G Modèle ML sur les 17 catégories · 8H Textes, écran « À propos », dossier Google Play	À faire

4. Résultats mesurés

Chiffres issus de `docs/generated/STATUS.md`, fichier généré par `tools/verify.ps1` lors de la dernière vérification (aucun chiffre recopié à la main).

Indicateur	Valeur
Tests automatisés (xUnit)	231 / 231 réussis
Build Android Release	Réussi
Garde-fous (aucun `catch {}` vide, aucun appel bloquant, permissions, aucun secret suivi par Git)	Tous réussis
Catégorisation : règles seules (176 dépenses de validation)	143 / 176 (81,25 %)
Catégorisation : modèle ML.NET seul	161 / 176 (91,48 %)
Catégorisation : pipeline hybride (mémoire + règles + ML)	175 / 176 (99,43 %)
Taille du modèle embarqué	108,6 Ko
Test automatique sur émulateur Android (CI, lot 8D)	14 / 14 étapes

Limite honnête : le jeu de validation est étiqueté sur 11 anciennes catégories et non sur les 17 actuelles ; le chiffre de 99,43 % sera recalculé au lot 8G. Le test sur émulateur couvre le lot 8D ; le lot 8E n'est pas encore passé en CI.

5. Difficultés rencontrées

- **Écart entre l'interface et le code réel.** L'audit a trouvé des écrans de démonstration : tontines avec des membres fictifs codés en dur, précision du modèle affichée en dur, export qui n'écrivait aucun fichier. Il a fallu tout reconstruire sur la base de données et ajouter un test qui vérifie que chaque écran est réellement accessible par un lien.
- **Chiffrement et clé.** Une clé de secours était écrite en clair si le coffre du système échouait, et la base devenait illisible si la clé changeait. Solution : plus aucun repli en clair, et un écran dédié qui laisse l'utilisateur décider en cas de problème.
- **Performance du code PIN.** Le hachage PBKDF2 à 600 000 itérations bloquait l'interface ; il a été déplacé hors du fil de l'interface.
- **Intégration continue.** La CI n'avait jamais tourné et le script d'émulateur était invalide. Elle a été réécrite avec un auto-test embarqué qui ne s'exécute que dans la version de test.
- **Sauvegarde portable.** Une sauvegarde chiffrée avec la clé du téléphone était inutilisable sur un autre appareil. Elle est désormais chiffrée par un mot de passe choisi par l'utilisateur, avec double confirmation et retour arrière automatique avant toute restauration.
- **Contexte camerounais.** Données d'entraînement d'abord américaines, revenus irréguliers non gérés, et choix de ne jamais lire les SMS (règle Google Play) : l'import SMS se fera par copier-coller ou partage.
- **Environnement de travail.** Migration de .NET 8 (plus supporté) vers .NET 10, problèmes d'encodage des fichiers sous PowerShell 5.1, et installation de l'application Windows impossible sans le mode développeur.

6. Reste à faire

- Lots 8F (import SMS réel), 8G (modèle sur les 17 catégories, jeu anglais) et 8H (textes restants, écran « À propos », avertissement « pas un conseil financier », dossier Google Play).
- Tests sur un vrai téléphone Android (ARM) selon `docs/DEVICE_TEST_CHECKLIST.md`, clé de signature et publication.